- CGI AWS Integration Architect — L2 Technical Deep-Dive (Part 1)
- AWS Integration Architecture + EKS IAM + Security
  - SECTION 1: EKS IAM ROLES — THE DEEP DIVE
    - Q1. How are IAM roles designed for EKS workloads? Explain IRSA in detail.
    - Q2. What is EKS Pod Identity and how does it differ from IRSA?
    - Q3. How do you design IAM for a multi-namespace EKS cluster with different teams?
    - Q4. How do you handle cross-account access from EKS pods?
  - SECTION 2: ADVANCED AWS INTEGRATION ARCHITECTURE
    - Q5. Design a real-time retail order integration spanning OMS, Salesforce, ERP, and payment gateway.
    - Q6. How do you design an integration architecture for migrating from on-prem ESB to AWS?
    - Q7. How do you handle API throttling and rate limiting at scale?
    - Q8. How do you implement the Saga pattern for distributed transactions in AWS?
    - Q9. How do you design for data consistency across Salesforce, OMS, and AWS?
    - Q10. How do you implement CI/CD and IaC for integration workloads?

CGI AWS Integration Architect — L2 Technical Deep-Dive (Part 1)

AWS Integration Architecture + EKS IAM + Security

> **Role:** AWS Integration Architect | **Round:** L2 Technical (1 Hour) **Focus:** Advanced AWS architecture, EKS IAM (IRSA/Pod Identity), hybrid integration, retail scenarios

---

SECTION 1: EKS IAM ROLES — THE DEEP DIVE

> *This is the area you specifically flagged. These questions are very likely in L2.*

Q1. How are IAM roles designed for EKS workloads? Explain IRSA in detail.

Answer:

In EKS, the core problem is: **how does a pod running inside Kubernetes get AWS permissions without using long-lived access keys?** The answer is **IRSA — IAM Roles for Service Accounts**.

How IRSA works (step-by-step):

1. **EKS creates an OIDC (OpenID Connect) provider** — When you create an EKS cluster, AWS provisions an OIDC identity provider. This establishes trust between Kubernetes service accounts and AWS IAM.

2. **You create an IAM role with a trust policy** that trusts the EKS OIDC provider:

```
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::123456:oidc-provider/oidc.eks.us-east-
1.amazonaws.com/id/EXAMPLED539D4633E53DE1B71"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "oidc.eks.us-east-1.amazonaws.com/id/EXAMPLED539D4633E53DE1B71:sub":
"system:serviceaccount:payments:payment-processor",
      "oidc.eks.us-east-1.amazonaws.com/id/EXAMPLED539D4633E53DE1B71:aud":
"sts.amazonaws.com"
    }
  }
}
```

The `Condition` block is critical — it restricts the role to **only** the `payment-processor` service account in the `payments` namespace. No other pod can assume this role.

3. **You annotate the Kubernetes ServiceAccount** with the IAM role ARN:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: payment-processor
  namespace: payments
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456:role/payment-processor-
role
```

4. **When a pod using this ServiceAccount starts**, the EKS Pod Identity Webhook mutates the pod spec to inject:
    - An AWS Web Identity Token file mounted at `/var/run/secrets/eks.amazonaws.com/serviceaccount/token`
    - Environment variables: `AWS_ROLE_ARN` and `AWS_WEB_IDENTITY_TOKEN_FILE`
5. **AWS SDK in the pod** automatically detects these environment variables and calls `sts:AssumeRoleWithWebIdentity` to get temporary credentials. No access keys stored anywhere.

Why this matters for security:

- **No long-lived credentials** — Tokens are short-lived (default 12 hours, configurable down to 15 minutes)
- **Least privilege per microservice** — Payment service gets DynamoDB access; notification service gets SES access. Neither can access the other's resources
- **Blast radius containment** — If a pod is compromised, the attacker only gets that pod's scoped permissions, not node-level permissions
- **Auditability** — CloudTrail shows exactly which service account assumed which role

Common mistake to avoid: Using the **EC2 instance profile** (node role) for all pods. This means every pod on that node gets the same permissions — a massive security anti-pattern. IRSA eliminates this entirely.

Q2. What is EKS Pod Identity and how does it differ from IRSA?

Answer:

EKS Pod Identity (launched late 2023) is AWS's simplified alternative to IRSA. Both solve the same problem — giving pods AWS permissions — but Pod Identity is operationally simpler.

Key differences:

Aspect	IRSA	EKS Pod Identity
Setup complexity	Requires OIDC provider + trust policy per role with specific audience/subject conditions	Install Pod Identity Agent add-on + create association via API
Trust policy	Complex — must include OIDC provider ARN, namespace, and SA name in conditions	Simple — trust `pods.eks.amazonaws.com` service principal
Cross-account	Complex — OIDC provider must be registered in target account	Simpler — standard role chaining works
Token management	Web Identity Token projected into pod	EKS Agent handles credential vending
Scalability	Each role needs unique trust policy referencing OIDC URL	Same trust policy pattern across clusters

Pod Identity trust policy (much simpler):

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "pods.eks.amazonaws.com"
  },
  "Action": ["sts:AssumeRole", "sts:TagSession"]
}
```

Then create a pod identity association:

```
aws eks create-pod-identity-association \
  --cluster-name my-cluster \
  --namespace payments \
```

```
  --service-account payment-processor \
  --role-arn arn:aws:iam::123456:role/payment-processor-role
```

When to use which:

- **New projects:** Prefer Pod Identity — simpler setup, easier cross-account, less error-prone
- **Existing IRSA deployments:** No urgency to migrate — IRSA still works and is fully supported
- **Self-managed K8s (non-EKS):** IRSA concepts (OIDC federation) still apply; Pod Identity is EKS-only

---

Q3. How do you design IAM for a multi-namespace EKS cluster with different teams?

Answer:

Scenario: An EKS cluster hosts 4 teams — payments, inventory, notifications, analytics. Each team must only access their own AWS resources.

IAM design:

```
EKS Cluster
├── Namespace: payments
│   └── ServiceAccount: payment-svc → IAM Role: payment-role
│       ├── DynamoDB: Orders table (read/write)
│       ├── Secrets Manager: payment-gateway-key (read)
│       └── SQS: payment-queue (send/receive)
│
├── Namespace: inventory
│   └── ServiceAccount: inventory-svc → IAM Role: inventory-role
│       ├── DynamoDB: Inventory table (read/write)
│       ├── S3: inventory-data bucket (read/write)
│       └── SNS: stock-alerts topic (publish)
│
├── Namespace: notifications
│   └── ServiceAccount: notification-svc → IAM Role: notification-role
│       ├── SES: SendEmail (allow)
│       ├── SNS: notification topics (publish)
│       └── SQS: notification-queue (receive)
│
└── Namespace: analytics
    └── ServiceAccount: analytics-svc → IAM Role: analytics-role
        ├── S3: data-lake bucket (read-only)
```

```
        ├── Athena: RunQuery (allow)
        └── Glue: GetTable, GetDatabase (read-only)
```

Enforcement layers:

1. **IRSA/Pod Identity** — Each service account maps to a scoped IAM role (as above)
2. **Kubernetes RBAC** — Teams can only deploy to their own namespaces
3. **Network Policies** — Pods in `payments` namespace cannot communicate with `analytics` pods
4. **OPA Gatekeeper** — Admission controller ensures pods must use a service account with IRSA annotation; bare pods without proper SA are rejected
5. **Resource Quotas** — Each namespace has CPU/memory limits to prevent resource hogging

Node-level role (minimal): The EC2 instance profile (node role) gets only:

- `ecr:GetDownloadUrlForLayer`, `ecr:BatchGetImage` — pull container images
- `ec2:DescribeInstances` — node registration
- `eks:DescribeCluster` — cluster discovery
- **Nothing else.** All workload permissions go through IRSA/Pod Identity.

Q4. How do you handle cross-account access from EKS pods?

Answer:

Scenario: Payment microservice in Account A (EKS cluster) needs to write to a DynamoDB table in Account B (shared data account).

Solution — Role chaining with IRSA:

Step 1: Pod assumes its local role (Account A) via IRSA:

```
payment-processor SA → arn:aws:iam::ACCOUNT_A:role/payment-pod-role
```

Step 2: Local role has permission to assume a role in Account B:

```
{
  "Effect": "Allow",
  "Action": "sts:AssumeRole",
  "Resource": "arn:aws:iam::ACCOUNT_B:role/cross-account-dynamodb-writer"
}
```

Step 3: Account B role trusts Account A role:

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::ACCOUNT_A:role/payment-pod-role"
  },
  "Action": "sts:AssumeRole"
}
```

Step 4: Application code:

```
sts = boto3.client('sts')
credentials = sts.assume_role(
    RoleArn='arn:aws:iam::ACCOUNT_B:role/cross-account-dynamodb-writer',
    RoleSessionName='payment-processor'
)
dynamodb = boto3.resource('dynamodb',
    aws_access_key_id=credentials['Credentials']['AccessKeyId'],
    aws_secret_access_key=credentials['Credentials']['SecretAccessKey'],
    aws_session_token=credentials['Credentials']['SessionToken']
)
```

SECTION 2: ADVANCED AWS INTEGRATION ARCHITECTURE

Q5. Design a real-time retail order integration spanning OMS, Salesforce, ERP, and payment gateway.

Answer:

```
Customer (Web/Mobile)
  │
  ▼
API Gateway (REST) + WAF + Cognito Auth
  │
  ▼
Lambda: Order Validation (inventory check, fraud score)
  │
  ▼
Step Functions: Order Orchestration Workflow
  ├─► State 1: Payment Authorization
  │     └─ Lambda → Payment Gateway (Stripe) via PrivateLink
  │        ├─ Success → next state
  │        └─ Fail → Compensating: notify customer, log for retry
  │
  ├─► State 2: Inventory Reservation
  │     └─ Lambda → DynamoDB (conditional write: decrement stock)
  │        ├─ Success → next state
  │        └─ Fail → Compensating: reverse payment auth
  │
  ├─► State 3: OMS Order Creation
  │     └─ Lambda → OMS API (create fulfillment order)
  │
  ├─► State 4 (Parallel):
  │     ├─ Lambda → Salesforce (create case via AppFlow/REST)
  │     ├─ Lambda → ERP (SAP IDoc via Amazon MQ bridge)
  │     └─ Lambda → SES (order confirmation email)
  │
  └─► State 5: EventBridge (order.confirmed event)
        ├─ Target: SQS → Warehouse Lambda (pick/pack instruction)
        ├─ Target: Lambda → Analytics pipeline (S3 data lake)
        └─ Target: SNS → Partner notifications
```

Key design decisions:

- **Step Functions** for the core workflow — provides visual debugging, built-in retry, and compensating transactions (Saga pattern)
- **EventBridge** post-confirmation for fan-out to downstream consumers — decoupled, new consumers can subscribe without changing the workflow
- **DynamoDB conditional writes** for inventory — prevents overselling under concurrent requests
- **Idempotency** at every step — Step Functions provides exactly-once execution; Lambda functions use idempotency keys in DynamoDB

Q6. How do you design an integration architecture for migrating from on-prem ESB

to AWS?

Answer:

Current state: Enterprise Service Bus (MuleSoft/TIBCO/IBM Integration Bus) handling 200+ integrations with SOAP, JMS, and file-based patterns.

Migration strategy — Strangler Fig Pattern:

Phase 1 — Coexistence (Month 1-3):

```
On-Prem ESB ──── Direct Connect ──→ AWS Integration Layer
                                      ├─ API Gateway (new APIs)
                                      ├─ EventBridge (new events)
                                      └─ Amazon MQ (JMS bridge)
```

- Stand up AWS integration services alongside ESB
- New integrations go directly to AWS
- Amazon MQ bridges JMS messages from on-prem to AWS

Phase 2 — Migration (Month 4-9):

- Migrate integrations in batches, grouped by domain (orders, inventory, customers)
- Per integration:
  - SOAP → REST API (API Gateway + Lambda)
  - JMS queues → SQS/SNS
  - File drops → S3 + EventBridge notifications + Glue
  - ESB orchestrations → Step Functions

Phase 3 — Decommission (Month 10-12):

- All traffic routed through AWS
- ESB decommissioned
- Monitoring validates no residual traffic

ESB to AWS service mapping:

ESB Capability	AWS Equivalent
Message routing	EventBridge rules
Message transformation	Lambda / Step Functions

ESB Capability	AWS Equivalent
Message queuing	SQS / SNS
Orchestration	Step Functions
API management	API Gateway
Data transformation	Glue ETL
Protocol mediation (SOAP↔REST)	Lambda + API Gateway
Monitoring	CloudWatch + X-Ray

Q7. How do you handle API throttling and rate limiting at scale?

Answer:

Multi-layer throttling strategy:

Layer 1 — API Gateway:

- **Account-level:** 10,000 RPS default (can be increased)
- **Stage-level:** Set per-stage throttle (e.g., production: 5000 RPS, staging: 500 RPS)
- **Per-method:** `/orders POST` at 1000 RPS, `/products GET` at 5000 RPS
- **Usage Plans + API Keys:** Per-client limits — Partner A gets 500 RPS, Partner B gets 200 RPS

Layer 2 — Lambda Concurrency:

- **Reserved concurrency:** Payment Lambda = 100 concurrent (protects downstream payment gateway limit)
- **Provisioned concurrency:** 50 warm instances for latency-sensitive APIs (no cold starts)

Layer 3 — Downstream Protection:

- **SQS as buffer:** If downstream system (ERP/OMS) handles only 50 TPS, put SQS in front. Lambda processes at controlled rate using reserved concurrency
- **Step Functions with rate limiter:** Use Wait states or SQS-based back-pressure

Layer 4 — Application-level:

- **Token bucket algorithm** in Lambda for fine-grained rate limiting per tenant
- **DynamoDB counter** tracking per-tenant usage with TTL for sliding window

Handling 429 (Too Many Requests):

- Return `429` with `Retry-After` header
- Client SDK implements exponential backoff with jitter
- CloudWatch alarm on throttle count → investigate and adjust limits

---

Q8. How do you implement the Saga pattern for distributed transactions in AWS?

Answer:

Problem: An order involves Payment + Inventory + Shipping across different services. If shipping fails, you must reverse inventory reservation and refund payment. Traditional ACID transactions don't work across distributed services.

Solution — Orchestration-based Saga using Step Functions:

```
{
  "StartAt": "ProcessPayment",
  "States": {
    "ProcessPayment": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:...:ProcessPayment",
      "Catch": [{
        "ErrorEquals": ["PaymentFailed"],
        "Next": "NotifyCustomerFailed"
      }],
      "Next": "ReserveInventory"
    },
    "ReserveInventory": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:...:ReserveInventory",
      "Catch": [{
        "ErrorEquals": ["InsufficientStock"],
        "Next": "RefundPayment"
      }],
      "Next": "CreateShipment"
    },
    "CreateShipment": {
      "Type": "Task",
```

```
      "Resource": "arn:aws:lambda:...:CreateShipment",
      "Catch": [{
        "ErrorEquals": ["ShipmentFailed"],
        "Next": "ReleaseInventory"
      }],
      "Next": "OrderComplete"
    },
    "ReleaseInventory": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:...:ReleaseInventory",
      "Next": "RefundPayment"
    },
    "RefundPayment": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:...:RefundPayment",
      "Next": "NotifyCustomerFailed"
    },
    "NotifyCustomerFailed": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:...:NotifyFailure",
      "End": true
    },
    "OrderComplete": {
      "Type": "Succeed"
    }
  }
}
```

Why Step Functions over choreography (event-based Saga):

- **Visibility:** Step Functions gives you a visual execution graph — you see exactly where each order is
- **Compensating actions are explicit:** Each Catch block routes to the correct rollback
- **Debugging:** Click on any failed execution to see input/output at each state
- **Timeout handling:** Built-in `TimeoutSeconds` per state — if payment gateway doesn't respond in 30s, trigger compensation

---

Q9. How do you design for data consistency across Salesforce, OMS, and AWS?

Answer:

Challenge: Salesforce, OMS, and AWS each have their own data stores. A customer update in Salesforce must propagate to OMS and the AWS data lake without

inconsistency.

Pattern — Event-Driven Eventual Consistency:

```
Salesforce (source of truth for customer data)
  │
  ▼ Platform Event: customer.updated
  │
Amazon AppFlow (CDC - real-time)
  │
  ▼
EventBridge (central event bus)
  ├──► SQS → Lambda → OMS API (update customer in OMS)
  ├──► SQS → Lambda → DynamoDB (update customer cache)
  └──► Firehose → S3 (data lake - analytics)
```

Consistency guarantees:

- **Idempotent consumers:** Each Lambda checks `last_modified_timestamp` before applying update — prevents out-of-order overwrites
- **DLQ with alerting:** If OMS update fails, message goes to DLQ. Alarm triggers. Support team investigates and redrives
- **Reconciliation job:** Daily Glue job compares Salesforce ↔ OMS ↔ DynamoDB records, flags mismatches
- **Conflict resolution:** Last-writer-wins with timestamp comparison. For critical fields (email, phone), Salesforce is source of truth

---

Q10. How do you implement CI/CD and IaC for integration workloads?

Answer:

IaC approach:

```
infrastructure/
├── terraform/
│   ├── modules/
│   │   ├── api-gateway/     # Reusable API Gateway module
│   │   ├── lambda/          # Lambda with SQS, DLQ, alarms
│   │   ├── step-functions/  # Step Functions with IAM
│   │   ├── eventbridge/     # Event bus, rules, targets
│   │   └── networking/      # VPC, endpoints, security groups
```

```
│   ├── environments/
│   │   ├── dev/
│   │   ├── staging/
│   │   └── prod/
│   └── backend.tf           # S3 + DynamoDB state locking
```

CI/CD pipeline (GitHub Actions):

```yaml
# Simplified pipeline
on:
  push:
    branches: [main, develop]

jobs:
  test:
    - run: pytest tests/ -v          # Unit tests
    - run: cfn-lint templates/       # CloudFormation lint
    - run: tflint                    # Terraform lint
    - run: checkov -d terraform/     # Security scanning

  deploy-dev:
    needs: test
    - run: terraform plan -var-file=dev.tfvars
    - run: terraform apply -auto-approve
    - run: pytest integration_tests/ # Integration tests against dev

  deploy-staging:
    needs: deploy-dev
    - run: terraform apply -var-file=staging.tfvars
    - run: pytest e2e_tests/         # End-to-end tests

  deploy-prod:
    needs: deploy-staging
    environment: production          # Manual approval gate
    - run: terraform apply -var-file=prod.tfvars
    - run: smoke_tests.sh            # Post-deployment validation
```

Key practices:

- **Terraform modules** for every integration pattern — reusable, tested, versioned
- **State locking** with DynamoDB to prevent concurrent applies
- **Checkov/tfsec** for security scanning before any apply
- **Separate state files** per environment — blast radius isolation
- **Lambda versioning + aliases** — deploy new code to `$LATEST`, test, then shift alias from `v1` to `v2`

End of Part 1 — Continue to Part 2 for Kubernetes deep-dive questions with validated answers